Mastermind en Python

Compte rendu de SAE

Malkhasyan David

C1

18 octobre 2026

Table des matières

1 Introduction **2**

2 Découpage en fonctions **2**

3 Algorithme de calcul du résultat **2**

4 Programme principal **4**

5 Tests **5**

6 Difficultés rencontrées **6**

7 Améliorations **7**

8 Conclusion **7**

1 Introduction

Le **Mastermind** est un jeu qui consiste à trouver en 15 coups l'alignement des couleurs qui nous est inconnu. Il y a un total de 8 couleurs qu'on peut utiliser, le secret se trouve dans une ligne de 5 couleurs. Pour aider le joueur à gagner, à chaque proposition on lui dit combien de couleurs sont à la bonne place, et combien sont présentes dans la combinaison secrète mais à une autre position, pour qu'il se corrige, sachant qu'il est possible d'utiliser plusieurs fois la même couleur. Les pions mal placés seront en noir et ceux bien placés seront en blanc. Pour créer ce jeu, j'ai utilisé Python, la bibliothèque pygame et un fichier fourni avec le sujet pour la partie pygame (`mm.py`). L'objectif de cette SAE est de confronter le développeur à la requête d'un client, puis de développer des fonctions avec les quelques indications qui ont été données.

2 Découpage en fonctions

Fonction	Entrées	Sortie	Rôle
`generer_combinaison`	aucune	liste de 5 couleurs	tire la combinaison secrète
`calculer_resultat`	2 listes	tuple (bien, mal)	compare proposition et secrète
`est_gagne`	tuple	booléen	vrai si 5 bien placés
`manche_finie`	tuple, 2 entiers	booléen	vrai si gagné ou coups épuisés
`afficher_dialogue`	fenêtre, texte	rien	boîte de dialogue semi-transparente
`afficher_fin`	fenêtre, booléen, entier	rien	choisit et affiche le message de fin
`principale`	aucune	rien	programme principal

TABLE 1 – Fonctions du programme

J'ai découpé le programme en plusieurs fonctions pour éviter qu'il y ait des problèmes dans mon code. Imaginons que j'aie une seule fonction principale : je pourrais rencontrer des bugs beaucoup plus fréquemment, et il serait compliqué de trouver l'erreur sur 100 lignes. Avec des fonctions séparées, le programme principal ne fait qu'appeler les fonctions, ce qui cause moins d'erreurs récurrentes. Les tests sont aussi plus simples : je peux tester chaque fonction sans lancer le jeu. Enfin, une fonction peut être réutilisée : par exemple, `est_gagne` est utilisée à la fois dans `manche_finie` et dans `principale`.

3 Algorithme de calcul du résultat

```
1  def calculer_resultat(proposition, secrete):
2      bien_place = 0
3      for i in range(len(proposition)):
4          if proposition[i] == secrete[i]:
5              bien_place = bien_place + 1
6      commun = 0
7      for couleur in mm.TabCouleur:
8          commun = commun + min(proposition.count(couleur), secrete.count(
               couleur))
9      mal_place = commun - bien_place
10     return bien_place, mal_place
```

Les bien placés

Un pion est bien placé lorsqu'il a la même couleur que dans la combinaison secrète **et** qu'il est à la même position. J'initialise la variable `bien_place` à 0 : elle sert de compteur. La boucle parcourt les positions de 0 à 4 et compare `proposition[i]` avec `secrete[i]`, avec le même indice `i` des deux côtés. Lorsque les deux couleurs sont égales, on ajoute 1 au compteur. Chaque bien placé correspond à un pion **blanc** sur le plateau.

Les mal placés

La variable `commun` compte le nombre de couleurs que la proposition et la secrète ont en commun, sans regarder les positions. Sur le plateau, c'est le total des pions : blancs + noirs.

Pour la calculer, la boucle parcourt la liste `mm.TabCouleur`, qui contient les 8 couleurs du jeu. À chaque tour, on compte combien de fois la couleur apparaît dans la proposition (`proposition.count(couleur)`) et combien de fois elle apparaît dans la secrète (`secrete.count(couleur)`). On garde le plus petit de ces deux nombres grâce à `min`, et on l'ajoute à `commun`.

On garde le plus petit des deux nombres car si la secrète a 2 Rouges et que la proposition en a 4, seuls 2 Rouges peuvent correspondre.

Enfin, `commun` contient aussi les bien placés, donc on calcule `commun - bien_place` pour connaître le nombre de mal placés. Le résultat correspond au nombre de pions **noirs**.

Exemple

	pos 0	pos 1	pos 2	pos 3	pos 4
Secrète	Rouge	Rouge	Bleu	Vert	Noir
Proposition	Rouge	Bleu	Rouge	Rouge	Rouge

Bien placés : on compare les couleurs position par position. Seule la position 0 a la même couleur (Rouge) dans les deux combinaisons, donc `bien_place` $= 1$.

Couleurs en commun : pour chaque couleur, on garde le minimum entre la proposition et la secrète.

Couleur	dans la proposition	dans la secrète	minimum
Rouge	4	2	2
Bleu	1	1	1
Vert	0	1	0
Noir	0	1	0
Autres	0	0	0
Total : `commun`			**3**

Mal placés : `mal_place` $= 3 - 1 = 2$.

La fonction renvoie donc `(1, 2)` : un pion blanc et deux pions noirs.

Pourquoi une double boucle ne fonctionnait pas

Dans ma première version, j'utilisais deux boucles imbriquées qui comparaient chaque position de la proposition avec **toutes** les positions de la secrète. Prenons cet exemple :

	pos 0	pos 1
Secrète	Bleu	Rouge
Proposition	Rouge	Bleu

Faire une double boucle entraîne une erreur : une couleur est comptée comme bien placée dès qu'elle se trouve **n'importe où** dans la secrète. Par exemple, `proposition[0]` vaut Rouge : la deuxième boucle parcourt toutes les positions de la secrète et trouve que `proposition[0] == secrete[1]` (Rouge). On ajoute donc 1 à `bien_place`, alors que le Rouge n'est pas à la bonne position. Avec cet exemple, la double boucle trouve 2 bien placés au lieu de 0.

4 Programme principal

Étape du sujet	**Code**
a. afficher l'IHM	`mm.afficherPlateau, mm.afficherChoixCouleur`
b. combinaison secrète	`secrete = generer_combinaison()`
c. boucle de jeu	`while not manche_finie(...)`
i. proposition	`mm.construireProposition(fenetre, nb_coups)`
ii. résultat	`calculer_resultat(proposition, secrete)`
iii. affichage	`mm.afficherResultat(fenetre, resultat, nb_coups)`
d. message de fin	`mm.afficherSecret, afficher_fin`
e. fermeture	`boucle sur pygame.event.get()`

TABLE 2 – Correspondance entre le sujet et le code

Décalage des lignes du plateau

Lors du premier test, les cercles de la première proposition s'affichaient entre la case du secret et le plateau, au lieu d'être dans les cases. Toutes les lignes étaient décalées d'un cran vers le haut.

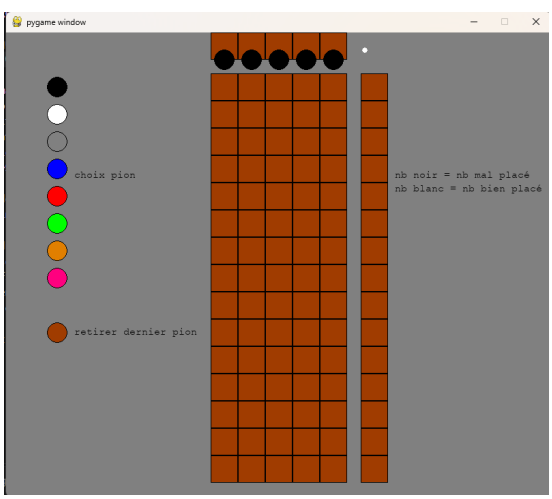
Ce décalage vient de `mm.py` : la fonction `afficherCombinaison` place les cercles à la hauteur $y = 40 + 40 \times (\text{ligne} - 1)$, alors que les cases du plateau commencent à $y = 60$. Résultat :

— avec `ligne = 1`, les cercles sont à $y = 40$, donc hors du plateau ;

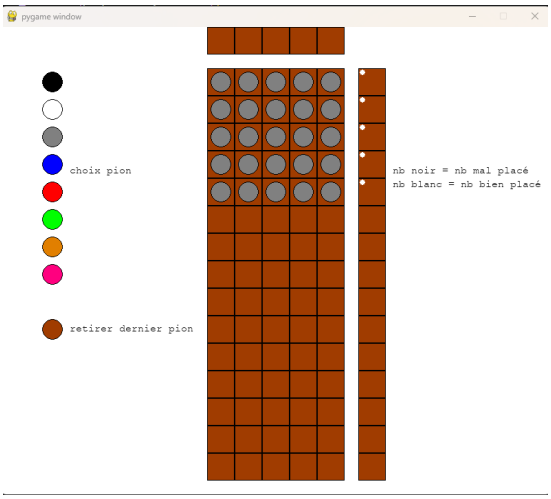
— avec `ligne = 2`, ils sont à $y = 80$, donc au milieu de la première case.

La ligne 1 de `mm.py` correspond donc à la zone du secret, et le plateau commence à la ligne 2.

Pour corriger ce problème, j'ai fait démarrer `nb_coups` à 1 au lieu de 0, et j'ai fixé `nb_coups_max` à 16. Comme `nb_coups` augmente au début de chaque tour, le premier coup s'affiche sur la ligne 2 et le dernier sur la ligne 16 : les 15 cases du plateau sont bien utilisées. En contrepartie, `nb_coups` a un coup d'avance sur le nombre réel de coups joués, c'est pourquoi j'envoie `nb_coups - 1` à `afficher_fin`.



(a) Avant : cercles hors des cases



(b) Après correction

FIGURE 1 – Décalage des lignes du plateau

5 Tests

```python
def test():
    print(generer_combinaison())

    # test 1 : tout pareil -> (5, 0)
    secrete     = [mm.Rouge, mm.Bleu, mm.Vert, mm.Noir, mm.Rose]
    proposition = [mm.Rouge, mm.Bleu, mm.Vert, mm.Noir, mm.Rose]
    print(calculer_resultat(proposition, secrete))

    # test 2 : 1 bien place, 2 mal places -> (1, 2)
    secrete     = [mm.Rouge, mm.Rouge, mm.Bleu, mm.Vert, mm.Noir]
    proposition = [mm.Rouge, mm.Bleu, mm.Rouge, mm.Rouge, mm.Rouge]
    print(calculer_resultat(proposition, secrete))

    # test 3 : rien en commun -> (0, 0)
    secrete     = [mm.Rouge, mm.Rouge, mm.Bleu, mm.Bleu, mm.Vert]
    proposition = [mm.Noir, mm.Blanc, mm.Gris, mm.Rose, mm.Orange]
    print(calculer_resultat(proposition, secrete))

    # test 4 : 5 bien places -> True (gagne)
    print(est_gagne((5, 0)))
    # test 5 : 3 bien places -> False (pas encore gagne)
    print(est_gagne((3, 1)))

    # test 6 : gagne au coup 3 -> True (manche finie)
    print(manche_finie((5, 0), 3, 10))
    # test 7 : pas gagne mais 10 coups sur 10 -> True (plus de coups)
    print(manche_finie((2, 1), 10, 10))
    # test 8 : pas gagne et 3 coups sur 10 -> False (on continue)
    print(manche_finie((2, 1), 3, 10))

```

FIGURE 2 – Résultat de l'exécution de la fonction `test`



(a) Victoire

(b) Défaite

FIGURE 3 – Fins de partie avec la boîte de dialogue et le secret affiché

6 Difficultés rencontrées

Le tableau suivant résume les principales erreurs rencontrées pendant le développement, leur cause et la correction apportée.

Problème	**Cause**	**Correction**
Plantage dans `generer_combinaison`	`liste.append[i]` : crochets au lieu de parenthèses, et ajout de l'indice `i` au lieu de la couleur	`liste.append(nouveau)`
Combinaison de 8 couleurs au lieu de 5	`range(len(mm.TabCouleur))` fait autant de tours qu'il y a de couleurs dans le jeu	constante `taille_combinaison = 5`
`TypeError` dans `calculer_resultat`	`range(proposition)` : `range` attend un nombre, pas une liste	`range(len(proposition))`
Nombre de bien placés faux	comparaison des indices `i` et `j` au lieu des couleurs, puis double boucle (voir section 3)	une seule boucle avec `proposition[i] == secrete[i]`
Nombre de mal placés faux	une couleur différente était comptée comme mal placée, même si elle n'existait pas dans la secrète	méthode `commun - bien_place`
Message « ÉCHEC » affiché même en cas de victoire	en remplaçant les `return` par des `print`, la fonction ne s'arrêtait plus et exécutait aussi la dernière ligne	message d'échec placé dans un `else` aligné avec `if gagne`
`TypeError: argument 1 must be Surface, not list`	`mm.afficherChoixCouleur (mm.TabCouleur)` : la liste des couleurs donnée à la place de la fenêtre	`mm.afficherChoixCouleur(fenetre)`
Légende du plateau coupée	fenêtre de 600 × 650 pixels trop petite pour le plateau et le texte	fenêtre de 800 × 680 pixels
Cercles hors des cases	décalage d'une ligne dans `mm.py`	voir section 4

TABLE 3 – Difficultés rencontrées

Ces erreurs m'ont appris plusieurs choses. D'abord, quand le programme plante, le message

d'erreur indique la ligne et la fonction où se trouve le problème : il suffit de le lire pour savoir où chercher. Ensuite, pour utiliser les fonctions de `mm.py`, j'ai regardé leur définition pour savoir quels paramètres leur donner, au lieu de deviner. Enfin, tester chaque fonction avec des exemples dont je connaissais déjà le résultat m'a permis de voir rapidement quand un calcul était faux, notamment pour `calculer_resultat`.

7 Améliorations

Améliorations réalisées

- **Message de fin dans la fenêtre** : au lieu d'afficher le résultat dans le terminal avec `print`, la fonction `afficher_dialogue` dessine une boîte de dialogue en bas de la fenêtre, inspirée des jeux Pokémon.
- **Fond semi-transparent** : la boîte utilise une surface `pygame.SRCALPHA` remplie d'un blanc partiellement transparent, ce qui laisse deviner le plateau derrière.
- **Affichage de la combinaison secrète** à la fin de la partie, avec `mm.afficherSecret`, pour que le joueur voie la solution.
- **Messages différents selon le nombre de coups** : un message spécial en 1 coup, un autre en 5 coups ou moins, et un dernier au-delà.

Améliorations possibles

- Faire apparaître le texte de la boîte de dialogue lettre par lettre, comme dans Pokémon.
- Proposer de rejouer une partie sans relancer le programme.
- Ajouter un mode sans doublons dans la combinaison secrète, avec `random.sample`.
- Afficher le nombre de coups restants pendant la partie.

8 Conclusion

Tout au long du projet, j'ai créé un Mastermind complet et jouable en local. J'ai aussi effectué une modification personnelle : une boîte de dialogue inspirée de Pokémon pour afficher le message de fin, avec un fond transparent. J'ai mis du temps au début à cause de la fonction `calculer_resultat`. Ce projet m'a permis de découvrir ce qu'était pygame et quelques fonctionnalités pour l'utiliser, même si je ne les connais pas toutes. J'ai également appris à utiliser les tuples et à découper un programme en fonctions. Enfin, j'ai appris à rédiger un rapport avec LaTeX, qui est très pratique et agréable visuellement. Si j'avais eu plus de temps, j'aurais pu faire apparaître le texte de la boîte de dialogue lettre par lettre, comme dans Pokémon.